

DiFlow-TTS: Compact and Low-Latency Zero-Shot Text-to-Speech with Discrete Flow Matching

Ngoc-Son Nguyen¹, Thanh V. T. Tran¹, Hieu-Nghia Huynh-Nguyen¹, Truong-Son Hy², Van Nguyen^{1,†}

¹ FPT Software AI Center, Vietnam

² University of Alabama at Birmingham, USA

{sonnn45, thanhvt1, vannth19}@fpt.com, huynhnghienhieu1999@gmail.com, thy@uab.edu

🔗 Code: <https://github.com/Fsoft-AIC/DiFlowTTS>

Abstract

Zero-shot text-to-speech (TTS) has made significant progress in replicating unseen voices, yet balancing generation quality and inference efficiency remains challenging. Autoregressive models suffer from high latency, while diffusion-based approaches are constrained by training-time configurations. Moreover, most flow-based methods operate in continuous space, which introduces optimization challenges because continuous token spaces are inherently more complex than discrete ones. To address these limitations, we propose **DiFlow-TTS**, a novel zero-shot TTS framework based on discrete flow matching. The model consists of a deterministic Phoneme-Content Mapper for linguistic modeling and a Factorized Discrete Flow Denoiser that simultaneously generates prosody and acoustic token streams. Experimental results demonstrate the effectiveness of our approach across multiple evaluation metrics.

Index Terms: text-to-speech synthesis, discrete flow matching, factorized discrete flow

1. Introduction

Zero-shot text-to-speech (TTS) has made remarkable progress in recent years, with the goal of generating high-quality speech that faithfully replicates the voice of previously unseen speakers from only a few seconds of reference audio. Neural audio codecs [1, 2] have enabled high-fidelity audio compression by efficiently encoding raw waveforms into discrete tokens while preserving quality through the residual vector quantization (RVQ) method. Recent studies have explored autoregressive (AR) modeled by applying language modeling techniques to these discrete tokens [3, 4, 5, 6], demonstrating high-quality speech generation. Although these methods are widely validated for discrete tokens, they inherently suffer from high inference latency due to their step-by-step sequential generation.

To address this issue, recent efforts to adapt these discrete codec tokens to non-autoregressive generation have sparked growing interest in applying diffusion models in both discrete settings [7, 8, 9, 10] and continuous settings [11, 12]. However, diffusion-based methods tightly couple the training and sampling processes, restricting sampling configurations to those established during training [13]. Modifying components such as noise schedules or rate matrices requires retraining for each new configuration, resulting in considerable computational overhead. Flow matching, on the other hand, offers a more flexible and efficient alternative. Most existing flow-based models still operate in purely continuous representation spaces and follow a single design paradigm, where flow is defined over continuous representations derived from discrete inputs [14] or mel-spectrograms

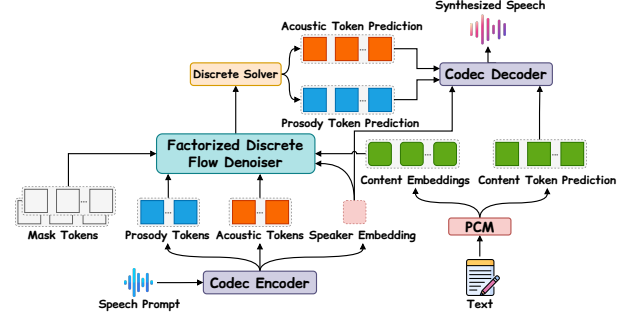


Figure 1: **Overview of DiFlow-TTS.** A *Codec Encoder* decomposes the speech prompt into a *speaker embedding*, *prosody*, and *acoustic tokens* for zero-shot style transfer, while the *Phoneme-Content Mapper* converts text into *content tokens* and *content embeddings*. Conditioned on the content embeddings and the speaker identity, prosody, and acoustic tokens extracted from the speech prompt, the *Factorized Discrete Flow Denoiser* simultaneously generates *prosody* and *acoustic tokens*. Finally, the generated tokens together with the speaker embedding are fed into the *Codec Decoder* to reconstruct the waveform.

[15, 16]. Although these representations are theoretically richer, they span a highly dimensional and unbounded space, which complicates density estimation and increases the likelihood of out-of-distribution artifacts, unlike discrete tokens that exist within a structured and finite space [17]. This limitation motivates our exploration of Discrete Flow Matching (DFM) [18] for speech synthesis.

In response, we present **Discrete Flow Matching** for zero-shot **TTS** (DiFlow-TTS), illustrated in Figure 1, a novel zero-shot TTS framework that leverages DFM tailored to discrete settings in speech synthesis. This work aims to demonstrate DFM as a promising research direction for speech synthesis, supported by empirical experiments that reveal both the advantages and limitations of this approach. To address this gap, we take an initial step toward developing a framework that operates directly in the discrete space of factorized codec tokens by employing a pre-trained FACodec [10] as the target discrete representation. This choice is motivated by two key factors: ❶ FACodec factorizes speech into prosody, content, and acoustic attributes, enabling flexible modeling of each attribute independently; and ❷ it is pre-trained on a large-scale multi-speaker corpus, providing a robust and reliable codec foundation for our target data. Building on this, DiFlow-TTS explicitly models these factorized attributes within a compact and unified framework. Specifically, we design **Phoneme-Content Mapper** (PCM), which directly maps phoneme sequences to discrete speech to-

[†]indicates the corresponding author.

kens that represent the content of the utterance. This module further generates content embeddings that align closely with the semantic structure of the speech. These embeddings, along with auditory attributes extracted from the reference speech prompt, are then used to condition a **F**actorized **D**iscrete **F**low **D**enoiser (FDFD) module, allowing it to effectively clone the reference’s speaking characteristics. Crucially, we design the denoiser with separate prediction heads for the probability velocity of distinct speech aspects, specifically prosody and acoustic details, allowing it to simultaneously learn multiple attribute-specific distributions within a unified flow architecture. In summary, our main contributions are as follows:

- We introduce DiFlow-TTS, a novel zero-shot TTS framework that learns probability flows directly in the discrete space of factorized codec tokens, establishing an initial baseline for applying DFM to speech generation.
- Unlike prior DFM applications that operate on homogeneous discrete sequences, we propose a reformulated DFM framework over a structured, factorized representation by introducing the Factorized Discrete Flow Denoiser. This design explicitly models individual speech attributes through a flow-prediction mechanism with dedicated heads for prosody and acoustic details. To our knowledge, this is the first work to decompose probability velocity fields across multiple discrete subspaces within a single discrete flow process.
- We show that DiFlow-TTS achieves promising results compared to baseline models in terms of naturalness, content accuracy, and prosody preservation, while retaining a compact architecture that is up to $11.7\times$ smaller and delivering low-latency inference with up to $34\times$ speedup over baselines, making it a promising candidate for deployment in resource-constrained and latency-critical environments.

2. Related Work

A growing trend in speech synthesis focuses on converting raw waveforms into discrete token representations using vector-quantized variational autoencoders (VQ-VAE), which was first introduced by [19] in the field of computer vision and later adapted to speech processing [20, 21]. These tokenized representations have demonstrated greater naturalness and robustness compared to conventional mel-spectrogram-based approaches. To effectively model sequences of discrete speech tokens, recent efforts have adapted large language models (LLMs) from the natural language processing domain [3, 4, 5, 6]. A notable example is VALL-E [5], which leverages a pre-trained neural codec to encode speech into discrete codec tokens and reformulates zero-shot TTS as a conditional codec language modeling task. During inference, it performs autoregressive continuation from the acoustic tokens of a speech prompt, enabling high-fidelity speaker-consistent voice synthesis.

Although AR models achieve impressive quality, they are inherently limited by slow inference speeds. This limitation has prompted a shift toward NAR paradigms [11, 10, 22, 12, 23]. For example, NaturalSpeech 2 [11] uses diffusion [24, 25] to generate discrete acoustic tokens as continuous features. Its successor, NaturalSpeech 3 [10], further factorizes speech into subspaces of content, prosody, and acoustic details, employing multiple diffusion models to independently capture various acoustic characteristics. In parallel, flow matching [26, 27] has gained attention as a promising generative technique, producing strong results in various domains. Yet, most existing speech-related flow matching applications operate in continuous spaces,

using either mel-spectrogram representations or continuous representations derived from discrete tokens, and predict flows over these representations rather than directly over discrete representations. [15, 28, 29, 30, 14].

An emerging line of research seeks to extend iterative refinement techniques to discrete spaces by modeling generation dynamics with Markov chains. Discrete-space generative models have already proven effective in domains such as natural language [31, 32], proteins [33, 34], vision [35, 32, 36], code [18], and graphs [13]. However, the application of DFM [18] to speech modeling, such as zero-shot TTS, remains largely unexplored. In this work, we propose a zero-shot TTS system with DFM, aiming to leverage the advantages of discrete modeling while preserving high-quality speech generation.

3. Methodology

Figure 2 illustrates the details of the DiFlow-TTS framework, which consists of two core modules: **(b) Phoneme-Content Mapper** and **(c) Factorized Discrete Flow Denoiser**, described in detail in Sections 3.4 and 3.5, respectively. We first employ a pre-trained **(a) Speech Tokenizer**, described in Section 3.2, to extract discrete token sequences as our target representations, which are subsequently modeled by these two modules.

3.1. Notation

Let a sequence x be an array of L tokens $(x_1, x_2, \dots, x_L)$ drawn from a discrete vocabulary of size v , i.e., $x \in [v]^L$ with $[v] = \{1, \dots, v\}$. To represent point mass distributions over these sequences, we use the delta function $\delta_y(x) = \prod_{i=1}^L \delta_{y_i}(x_i)$, where $y \in [v]^L$; $\delta_{y_i}(x_i) = 1$ if $x_i = y_i$, and 0 otherwise. Building on this, we further define the extended space $[v]^{nL}$ to represent the concatenation of n sequences, each of length L .

3.2. Input Data Extraction

We employ FAcCodec [10], comprising a Codec Encoder and a Codec Decoder, as our *Speech Tokenizer* (Figure 2a) that converts raw speech waveforms into discrete token sequences serving as target representations in our framework. Specifically, the Codec Encoder converts the original speech signal $\mathbf{r}$ into disentangled token sequences representing prosody, content, and acoustic details and extracts the speaker embedding:

$$\mathbf{r}^p, \mathbf{r}^c, \mathbf{r}^a, \mathbf{s} = \text{CodecEncoder}(\mathbf{r}), \quad (1)$$

where $\mathbf{r}^p \in [v]^L$, $\mathbf{r}^c \in [v]^{2L}$, and $\mathbf{r}^a \in [v]^{3L}$ denote the token sequences for prosody, content, and acoustic details, respectively, and $\mathbf{s} \in \mathbb{R}^{D_{\text{spk}}}$ is the speaker embedding. Here, L is the token sequence length, and prosody, content, and acoustic details use one, two, and three Residual Vector Quantization (RVQ) codebooks, respectively, each with a vocabulary size of $v = 1024$.

We design a model that generates these tokens ($\hat{\mathbf{r}}^p \in [v]^L$, $\hat{\mathbf{r}}^c \in [v]^{2L}$, $\hat{\mathbf{r}}^a \in [v]^{3L}$) and synthesizes the speech waveform $\hat{\mathbf{r}}$ by feeding the generated tokens together with the speaker identity $\mathbf{s}$ into the Codec Decoder:

$$\hat{\mathbf{r}} = \text{CodecDecoder}(\hat{\mathbf{r}}^p, \hat{\mathbf{r}}^c, \hat{\mathbf{r}}^a, \mathbf{s}). \quad (2)$$

3.3. Discrete Flow Matching for Factorized Codec Tokens

We adopt discrete flow matching as the generative backbone for modeling prosody and acoustic codec tokens. The goal is to transport source samples $\mathbf{x}_0 \sim p$ to target samples $\mathbf{x}_1 \sim q$. Following [18], we instantiate the source distribution p to assign all

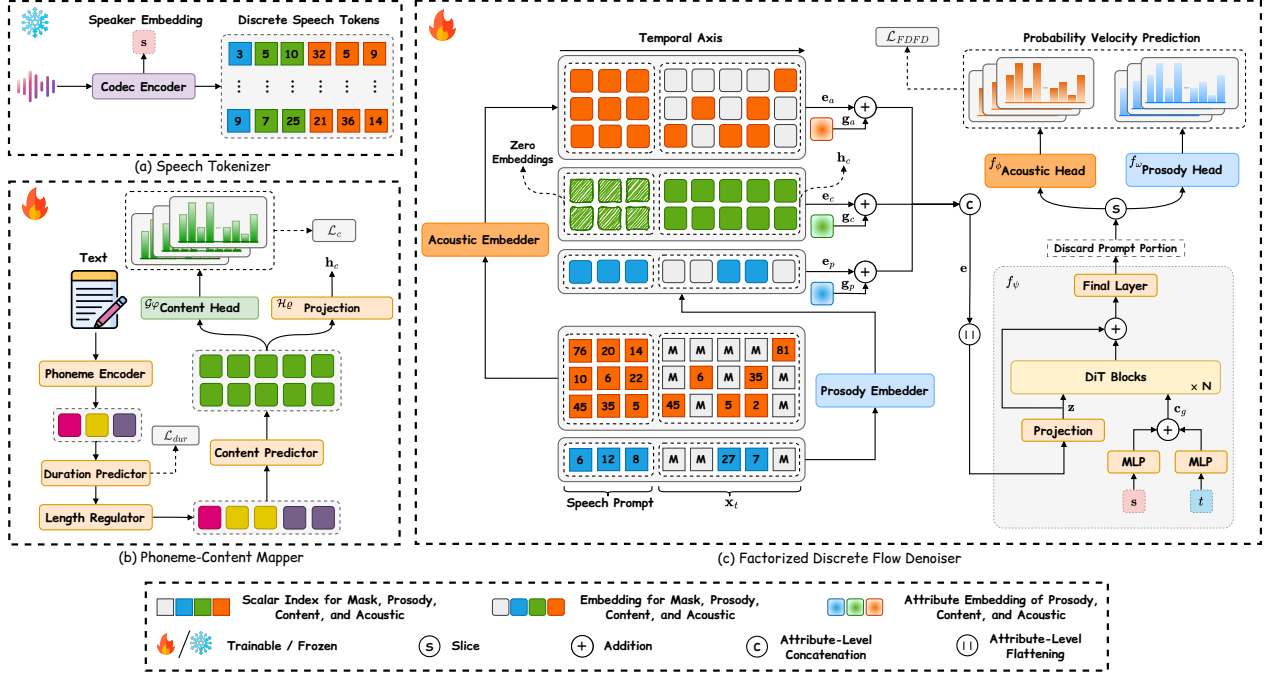


Figure 2: **Detailed architecture of DiFlow-TTS.** We formulate zero-shot TTS as token prediction over a factorized codec token space. Speech is tokenized by the (a) Speech Tokenizer into content, prosody, and acoustic tokens along with a speaker embedding. Built on these tokens, we design a framework comprising two main modules: (b) Phoneme-Content Mapper, which maps input phonemes to discrete content tokens and generates corresponding content embeddings; and (c) Factorized Discrete Flow Denoiser, which performs discrete flow matching conditioned on the speaker embedding, the discrete prosody and acoustic tokens derived from the speech prompt, and the content embeddings.

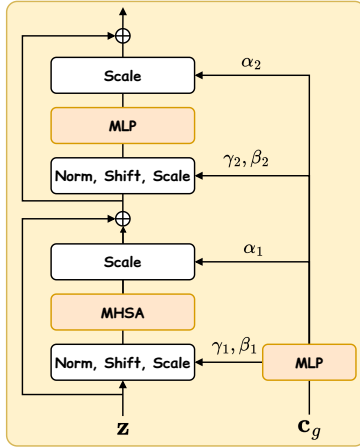


Figure 3: The detailed architecture of the DiT block.

probability mass to sequences in which every token is the mask token [MASK], that is, $p(x) = \delta_{[\text{MASK}]}(x)$. This implies that the source distribution places all probability mass in the sequence where every token is the mask token [MASK]. For the target distribution q , unlike prior DFM applications that operate on homogeneous discrete sequences, we reformulate the DFM framework by factorizing $\mathbf{x}_1$ into two structured components that are learned jointly. This structured formulation enables the construction of a probability velocity field over a composite target space consisting of these two components. Accordingly, we

define the target distribution q as follows:

Definition 3.1. Let $\mathbf{x}_1^p \sim q_p$ and $\mathbf{x}_1^a \sim q_a$ denote the random variables corresponding to the prosody and acoustic details sequences, respectively. These sequences are in spaces $[v]^L$ and $[v]^{3L}$. The full target sequence is then defined as $\mathbf{x}_1 = [\mathbf{x}_1^p; \mathbf{x}_1^a] \in [v]^{(1+3)L}$, where $[\cdot]$ denotes the concatenation of the sequence. Assuming the independence between the two components, the joint target distribution is factorized as $q(x) = q_p(x^p) \cdot q_a(x^a)$, where $x = [x^p; x^a]$.

During training, we employ a scheduler $\kappa_t \in [0, 1]$, a monotonically increasing function with boundary conditions $\kappa_0 = 0$ and $\kappa_1 = 1$, where $t \in [0, 1]$ denotes continuous time. This scheduler controls the interpolation, gradually shifting the distribution from source to target as κ_t increases. We follow the deterministic convex-interpolant assumption, under which the marginal path distribution is given by $p_t(\mathbf{x}^i | \mathbf{x}_t) = \delta_{\mathbf{x}_t}(\mathbf{x}^i)$. Following [18], we then construct a conditional probability path, referred to as the *mixture path*, which linearly interpolates between the source and target distributions: $p_t(\mathbf{x}^i | \mathbf{x}_t, \mathbf{x}_1) = (1 - \kappa_t)\delta_{\mathbf{x}_0}(\mathbf{x}^i) + \kappa_t\delta_{\mathbf{x}_1}(\mathbf{x}^i)$. This formulation leads to a *conditional probability path*, which is governed by the probability velocity $\mathbf{u}_t$ defined as:

$$\mathbf{u}_t^i(\mathbf{x}^i, \mathbf{x}_t) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[p_{1|t}(\mathbf{x}^i | \mathbf{x}_t, \mu; \theta) - \delta_{\mathbf{x}_t}(\mathbf{x}^i) \right], \quad (3)$$

where $\dot{\kappa}_t$ is the time derivative of the scheduler κ_t , θ denotes learnable parameters of a probability denoiser, $p_{1|t}(\cdot | \mathbf{x}_t, \mu; \theta)$ is the posterior distribution $\mathbf{x}_1$ given a partially corrupted sequence $\mathbf{x}_t$ and μ representing a set of multimodal conditioning inputs.

3.4. Phoneme-Content Mapper

The PCM module (Figure 2b) aligns phoneme sequences derived from the text prompt with discrete content tokens. While its overall structure is inspired by conventional duration-based alignment mechanisms [37], we reformulate the alignment process to operate directly on discrete codec tokens rather than continuous mel-spectrogram frames.

Given a text input, we convert it into a phoneme sequence and extract phoneme embeddings $\mathbf{p} \in \mathbb{R}^{N \times D}$ using a phoneme encoder, where N and D denote the sequence length and hidden dimension, respectively. To align phonemes with discrete speech tokens, a *Duration Predictor* estimates durations, indicating how many speech tokens correspond to each phoneme. This produces an integer-based alignment that maps each phoneme to a variable-length span in the speech-token sequence. Using these alignments, the *Length Regulator* upsamples phoneme embeddings based on the ground-truth (during training) or predicted (during inference) phoneme durations. The upsampled sequence, whose length is L , is then passed to the *Content Predictor*, which consists of 2 Feed-Forward Transformer (FFT) layers. These layers hierarchically extract textual representations at each level, producing hidden states $\mathbf{h} \in \mathbb{R}^{2 \times L \times D}$, which are then processed by two branches: a projection layer $\mathcal{H}_\theta(\cdot)$ produces content embeddings, and a content head $\mathcal{G}_\varphi(\cdot)$ that outputs logits over a vocabulary of size v :

$$\begin{aligned} \mathbf{h}_c &= \mathcal{H}_\theta(\mathbf{h}) \in \mathbb{R}^{2 \times L \times D}, \\ p(\mathbf{x}^c | \mathbf{h}; \varphi) &= \text{Softmax}(\mathcal{G}_\varphi(\mathbf{h})) \in \mathbb{R}^{2 \times L \times v}. \end{aligned} \quad (4)$$

3.5. Factorized Discrete Flow Denoiser

The FDFD (Figure 2c) aims to generate the prosody and acoustic sequences of the synthesized speech by leveraging DFM and in-context learning, conditioned on a set of contextual inputs. In the following, we detail the key elements of this module.

3.5.1. Contextual Modeling

We now elaborate on the construction of the conditioning context μ introduced in Eq. (3) and describe how it is integrated into our framework.

① Inputs: Given a reference speech prompt $\mathbf{r}$, we apply the operation in Eq. (1) to extract **prosody** (p) token sequences $\mathbf{r}^p \in [v]^{L_r}$, **acoustic** (a) token sequences $\mathbf{r}^a \in [v]^{3L_r}$, and a speaker embedding $\mathbf{s} \in \mathbb{R}^{D_{\text{spk}}}$, where L_r denotes the temporal length of the reference prompt and D_{spk} is the hidden dimension of the speaker embedding. The corrupted input at timestep t , $\mathbf{x}_t \in [v]^{(1+3)L}$, is split into prosody tokens $\mathbf{x}_t^p \in [v]^L$ and acoustic tokens $\mathbf{x}_t^a \in [v]^{3L}$. We then employ specific embedders for prosody and acoustic (implemented as learned embedding tables) to map each discrete token index from the vocabulary $[v]$ to D -dimensional hidden representations $\mathbf{e}_r^\alpha$ (**reference**) and $\mathbf{e}_t^\alpha$ (**corrupted**) for $\alpha \in \{p, a\}$.

② Conditioning: We concatenate the reference and corrupted embeddings along the temporal dimension for each attribute, where the reference embeddings provide contextual information for the corrupted embeddings. We further introduce the **content** (c) embedding (Eq. (4)) as the content conditioning signal ($\mathbf{e}_t^c = \mathbf{h}_c$) to provide linguistic guidance for prosody and acoustic modeling, while the reference content representation is replaced with zeros ($\mathbf{e}_r^c = \mathbf{0}$), since content is not transferred from the reference speech during style transfer. The concatenated embeddings for each attribute is given

by $\mathbf{e}_\alpha = [\mathbf{e}_r^\alpha; \mathbf{e}_t^\alpha] \in \mathbb{R}^{\beta \times (L_r + L) \times D}$, where $[\cdot]$ denotes the concatenation operator, and $(\alpha, \beta) \in \{(p, 1), (c, 2), (a, 3)\}$.

③ Constructing unified embedding: We first introduce a learnable attribute-type embedding $\mathbf{g}_\alpha \in \mathbb{R}^{1 \times 1 \times D}$ for each attribute $\alpha \in \{p, c, a\}$, allowing the model to explicitly differentiate among the corresponding attributes. Each such embedding is added to the embeddings of its corresponding attribute, yielding $\tilde{\mathbf{e}}_\alpha = \mathbf{e}_\alpha + \mathbf{g}_\alpha$. The resulting embeddings are concatenated along the attribute dimension, yielding the final unified embedding $\mathbf{e} = [\tilde{\mathbf{e}}_p; \tilde{\mathbf{e}}_c; \tilde{\mathbf{e}}_a] \in \mathbb{R}^{(1+2+3) \times (L_r + L) \times D}$.

3.5.2. Factorized Flow Prediction

We first flatten $\mathbf{e}$ along the attribute dimension, yielding a tensor $\in \mathbb{R}^{(L_r + L) \times (1+2+3) \times D}$, and then linearly project it to $\mathbf{z} \in \mathbb{R}^{(L_r + L) \times D}$. The resulting representation is processed by a neural network $f_\psi : \mathbb{R}^{(L_r + L) \times D} \rightarrow \mathbb{R}^{(L_r + L) \times (1+3) \times D}$ composed of Diffusion Transformer (DiT) blocks [38], a long skip connection, and a final transformation layer (as shown in Figure 3). To achieve speaker adaptation, we form a global conditioning vector $\mathbf{c}_g \in \mathbb{R}^D$ by summing two D -dimensional embeddings: the speaker embedding, obtained by projecting $\mathbf{s}$ through a Multi-Layer Perceptron (MLP), and the timestep embedding $\mathbf{t}$, obtained by mapping the timestep through an MLP. This vector modulates the DiT features via adaptive layer normalization (AdaLN) [38]. After processing the DiT blocks via AdaLN, we apply a long skip connection by adding $\mathbf{z}$ to the final DiT output, followed by the final transformation layer comprising layer normalization, AdaLN-based modulation conditioned on $\mathbf{c}_g$, and a linear projection to dimension $(1 + 3)D$. We then discard the reference portion and permute the result to yield the final hidden representation $\mathbf{h}_{p,a} \in \mathbb{R}^{(1+3) \times L \times D}$.

To effectively enable the model to jointly attend to information from different representation subspaces, we propose a factorized flow prediction mechanism based on multi-head prediction. In this design, FDFD simultaneously models multiple aspects of speech, specifically prosody and acoustic details. Formally, we define two parallel heads: the *prosody head* $f_\phi(\cdot)$ and the *acoustic head* $f_\omega(\cdot)$, which independently predict probability distributions corresponding to prosody and acoustic attributes. We first slice the representation $\mathbf{h}_{p,a}$ into two parts: ① the prosody representation $\mathbf{h}_p \in \mathbb{R}^{1 \times L \times D}$ and ② the acoustic representation $\mathbf{h}_a \in \mathbb{R}^{3 \times L \times D}$. Each part is processed by its respective head, $f_\phi(\cdot)$ and $f_\omega(\cdot)$, producing logits of the shapes $\mathbb{R}^{1 \times L \times v}$ and $\mathbb{R}^{3 \times L \times v}$, respectively. These logits correspond to the categorical distributions predicted over the discrete token vocabulary for each attribute. Finally, we concatenate the two outputs along the attribute dimension to obtain a tensor in $\mathbb{R}^{(1+3) \times L \times v}$ that serves as the estimated posterior distribution over $\mathbf{x}_1$.

3.6. Training Objective

The overall training objective consists of three loss components corresponding to different modules in our framework. First, we optimize the *Duration Predictor* using the Mean Squared Error (MSE) loss on the logarithmic scale, denoted as $\mathcal{L}_{dur}$, which compares the predicted and ground-truth durations. Second, for the *Content Predictor* defined in Eq. (4), we use a cross-entropy loss $\mathcal{L}_c$ between the predicted logits and the discrete content tokens obtained from the ground truth. Third, for the *FDFD* module, we learn a probabilistic denoiser $p_{1|t}$ trained to recover masked tokens under varying masking ratios. The objective is to

minimize the cross-entropy loss:

$$\mathcal{L}_{FDFD} = - \sum_{i \in \mathcal{T}} \mathbb{E}_{t \sim \mathcal{U}[0,1], (\mathbf{x}_0, \mathbf{x}_1), \mathbf{x}_t} \left[\log p_{1|t}(\mathbf{x}_1^i | \mathbf{x}_t, \mu; \theta) \right],$$

where $\mathcal{T} = [(1+3)L]$, $\mathbf{x}_t \sim p_t(\mathbf{x} | \mathbf{x}_0, \mathbf{x}_1)$, $\mathbf{x}_0 \sim p$, and $\mathbf{x}_1 \sim q$. Finally, the total loss is defined as:

$$\mathcal{L} = \lambda_{dur} \mathcal{L}_{dur} + \lambda_c \mathcal{L}_c + \lambda_{FDFD} \mathcal{L}_{FDFD}, \quad (5)$$

where λ_{dur} , λ_c , and λ_{FDFD} are hyperparameters that control the relative importance of each loss term.

4. Experimental Setup

4.1. Implementation Details

The PCM has a hidden dimension of 768 and integrates a variance adapter [37] for the Duration Predictor with an encoder hidden size of 256, a filter size of 1024, a kernel size of 3, and a dropout rate of 0.5. The Phoneme Encoder consists of 2 FFT layers with 4 attention heads, a hidden size of 256, an output dimension of 768, convolutional filter sizes of 1024 with kernel sizes [9, 1], a dropout rate of 0.2, and a maximum sequence length of 5000. The Content Predictor comprises multiple layers, each responsible for extracting textual representations for a corresponding content codebook level. Each layer shares the same FFT-based architecture as the Phoneme Encoder. The FDFD employs a scheduler defined as $\kappa_t = t^2$ and is built on DiT blocks [38] with a hidden size of 768, 12 layers and 12 attention heads, further enhanced with rotary position embedding (RoPE) [39]; the feed-forward network uses a width multiplier of 4. The embedding dimension of the speaker D_{spk} is 256.

We train the model on $4 \times$ NVIDIA A100 GPUs for 315K steps with a batch size of 16, using AdamW [40] with a learning rate of 1×10^{-4} , weight decay of 0.01, and 200K warm-up steps. For Eq. (5), the overall objective combines duration, content, and denoising losses, weighted by $\lambda_{dur} = 0.5$, $\lambda_c = 1.0$, and $\lambda_{FDFD} = 1.0$, respectively.

4.2. Baselines

To ensure a best-effort comparison under **publicly available checkpoints and code** with standard evaluation settings, we compare against publicly available baselines spanning different modeling paradigms: (i) *Autoregressive models*: VoiceCraft [3], VALL-E [5]; (ii) *Continuous flow matching/diffusion models*: NaturalSpeech 2 [11], F5-TTS [16], OZSpeech [14]; (iii) *Masked generative model*: MaskGCT [41]; (iv) *Discrete flow matching model*: DiFlow-TTS. We provide detailed descriptions of the training data, checkpoints, and evaluation settings below:

- **VoiceCraft**: We use the official code and the pre-trained checkpoint¹, trained on 9K hours of the GigaSpeech dataset [42].
- **F5-TTS**: We use samples obtained through communication with the authors of [14], reproduced using 500 hours of the LibriTTS dataset. We additionally perform inference using the official code² and a pre-trained checkpoint³ trained on 100K hours of the Emilia dataset [43].

¹https://huggingface.co/pypl/VoiceCraft/blob/main/830M_TSEnhanced.pth

²<https://github.com/SWivid/F5-TTS>

³<https://huggingface.co/SWivid/F5-TTS>

- **NaturalSpeech 2**: We use the Amphion toolkit [44] and the pre-trained weight⁴, trained on 585 hours of the LibriTTS dataset.
- **VALL-E**: We use samples provided through communication with the authors of [14]. The model is reproduced using the Amphion toolkit⁵ [44] and trained on 500 hours of the LibriTTS dataset.
- **OZSpeech**: We use samples provided through communication with the authors, trained on 500 hours of the LibriTTS dataset.
- **MaskGCT**: We use the official code and the pretrained checkpoint⁶, trained on English and Chinese data from Emilia [43], each with 50K hours of speech (totaling 100K hours). Since the baseline requires the total speech length, we use the ground-truth duration during inference.

4.3. Datasets

4.3.1. Training

We use a 470-hour subset of the LibriTTS dataset [45], which comprises multi-speaker English audio recordings, to train our method. We preprocess this dataset for training as follows:

The silent segments at the beginning and end of each utterance are removed. We retain audio clips ranging from 1.0 to 16.6 seconds in duration that contain utterances with more than three words. Speech is sampled at 16 kHz and tokenized using FAcCodec [10] at 80 tokens/s for ground-truth representations, which include a speaker embedding and six sequences of discrete tokens: one for prosody, two for content, and three for acoustic details. To obtain the ground truth of the text at the phoneme-level and the corresponding discrete speech tokens, we use the Montreal Forced Aligner (MFA) [46] to align each speech with its target transcription, producing the duration of each phoneme in the speech. We then multiply these durations by 80, which represents the number of tokens per second in FAcCodec, to determine the number of speech tokens corresponding to each phoneme.

4.3.2. Evaluation

We follow the evaluation setup used in OZSpeech [14] to assess our model on the LibriSpeech test-clean set [47]. This test set consists of 2.2 hours, each ranging from 4 to 10 seconds in length. We perform the cross-sentence generation task using two speech samples from the same speaker: one sample serves as the speech prompt, and the other provides the target text content to be synthesized.

4.4. Evaluation Metrics

To evaluate model performance, we employ a suite of **objective evaluation metrics** that assess multiple aspects of speech synthesis, including naturalness and speech quality (UTMOS), speaker similarity (SIM-O), content accuracy (WER), prosody accuracy and error (Pitch/Energy-Accuracy, Pitch/Energy-RMSE), and inference latency (RTF). Specifically, ① UTMOS [48] is a deep learning-based framework that predicts Mean Opinion Scores (MOS) to estimate speech naturalness and overall quality; ② for speaker similarity (SIM-O), we compute the cosine similarity between speaker embeddings extracted using a WavLM-TDNN speaker verification model [49], applied to the synthe-

⁴https://huggingface.co/amphion/naturalspeech2_libritts/tree/main/checkpoint

⁵<https://github.com/open-mmlab/Amphion>

⁶<https://huggingface.co/amphion/MaskGCT>

Table 1: Performance on the LibriSpeech test-clean dataset using 3-second audio prompts. $[\diamond]$ means reproduced results, $[\dagger]$ and $[\ddagger]$ mean results inferred from official and unofficial checkpoints, respectively. The best and second best are **bold** and underlined, respectively. WER is reported as a value in the range $[0, 1]$. Abbreviation: E (Emilia), GS (GigaSpeech), LT (LibriTTS).

Type	Model	Data (hours)	UTMOS $\uparrow$	WER $\downarrow$	SIM-O $\uparrow$	F0		Energy	
						Accuracy $\uparrow$	RMSE $\downarrow$	Accuracy $\uparrow$	RMSE $\downarrow$
-	Ground Truth	-	4.10	0.02	-	-	-	-	-
(i)	VoiceCraft $[\dagger]$ ACL'24	GS (9K)	3.55	0.18	0.51	0.78	17.22	0.44	<u>0.010</u>
	VALL-E $[\diamond]$ TASLPRO'25	LT (500)	3.68	0.19	0.40	0.75	21.66	0.36	0.020
(ii)	NaturalSpeech 2 $[\ddagger]$ ICLR'24	LT (585)	2.38	<u>0.09</u>	0.31	0.80	15.62	0.25	0.020
	F5-TTS $[\diamond]$ ACL'25	LT (500)	3.76	0.24	0.52	0.80	13.78	0.67	<u>0.010</u>
	F5-TTS $[\dagger]$ ACL'25	E (100K)	3.72	<u>0.09</u>	<u>0.66</u>	<u>0.83</u>	12.66	0.66	<u>0.010</u>
	OZSpeech $[\dagger]$ ACL'25	LT (500)	3.15	0.05	0.40	0.81	<u>11.96</u>	0.67	<u>0.010</u>
	MaskGCT $[\dagger]$ ICLR'25	E (100K)	<u>3.83</u>	<u>0.09</u>	0.67	0.77	14.33	0.75	0.007
(iv)	DiFlow-TTS (Ours)	LT (470)	3.98	0.05	0.45	0.88	7.97	<u>0.73</u>	0.007

Table 2: MOS evaluation with 3-second audio prompts, including 95% confidence intervals. The best and second best are **bolded** and underlined, respectively.

Model	Naturalness $\uparrow$	Intelligibility $\uparrow$	Similarity $\uparrow$
Ground Truth	4.42 $\pm$ 0.12	4.54 $\pm$ 0.11	4.29 $\pm$ 0.14
VoiceCraft	3.94 $\pm$ 0.17	4.08 $\pm$ 0.18	4.17 $\pm$ 0.15
VALL-E	3.71 $\pm$ 0.17	3.96 $\pm$ 0.17	3.99 $\pm$ 0.15
NaturalSpeech 2	2.62 $\pm$ 0.20	3.25 $\pm$ 0.21	2.63 $\pm$ 0.18
F5-TTS	3.97 $\pm$ 0.17	4.16 $\pm$ 0.14	4.07 $\pm$ 0.16
OZSpeech	2.80 $\pm$ 0.23	3.42 $\pm$ 0.24	3.20 $\pm$ 0.22
MaskGCT	<u>3.97 $\pm$ 0.16</u>	4.14 $\pm$ 0.15	<u>4.17 $\pm$ 0.15</u>
DiFlow-TTS (Ours)	4.18 $\pm$ 0.16	4.41 $\pm$ 0.13	4.42 $\pm$ 0.12

sized speech and the reference prompt; ③ for the word error rate (WER), we used the HuBERT-large model [50] (pre-trained on Libri-Light [51], and fine-tuned on 960h of LibriSpeech [47]) to transcribe the synthesized speech; ④ for prosody accuracy and error, we evaluate the alignment between synthesized speech and the reference prompt in terms of pitch (F0) and energy contours, where, following PromptTTS [52] and TextrolSpeech [53], F0 and energy values are discretized into three relative tiers (high, normal, and low) based on their mean values to compute tier-level accuracy (Pitch/Energy-Accuracy), and Root Mean Square Error (RMSE) is further calculated to quantify deviations in F0 (Pitch-RMSE) and energy (Energy-RMSE) between synthesized outputs and reference prompts; ⑤ for Real-Time Factor (RTF), we measure inference efficiency by quantifying the time required to generate one second of speech, with all RTF evaluations conducted in a fully end-to-end configuration on a single NVIDIA A100 80GB GPU.

Complementing these objective measures, we perform a **subjective evaluation** following the Mean Opinion Score (MOS) protocol. Specifically, 30 listeners rate the synthesized speech on a scale from 1 to 5 with respect to naturalness, intelligibility, and speaker similarity to the speech prompt.

5. Experimental Results

5.1. Main Results

5.1.1. Comparison Results

Table 1 presents the performance of DiFlow-TTS with 128 function evaluations (NFE) using 3-second audio prompts, compared to baseline methods. For naturalness and speech quality as measured by UTMOS, DiFlow-TTS achieves the strongest per-

formance despite being trained on only 470 hours of speech data, which is significantly smaller (by a factor of $1.1\times$ to $212.8\times$) than other baselines, highlighting the strength of our FDFD module in capturing prosodic and acoustic nuances even under limited data conditions. For content accuracy, DiFlow-TTS, along with OZSpeech, achieves the best performance in terms of WER, demonstrating the effectiveness of our method in producing speech with accurate linguistic content. For speaker similarity, DiFlow-TTS offers no clear advantage over baselines, likely due to its simple speaker conditioning in DiT blocks, which could be improved with more advanced strategies. **We highlight this limitation as a promising direction for future work beyond the scope of this study.** For prosody reconstruction, DiFlow-TTS outperforms across all metrics, with the sole exception of energy accuracy, where it trails MaskGCT by only 0.02, despite MaskGCT being trained on significantly more data (100K hours). These findings further confirm the ability of the FDFD module to model fine-grained prosodic attributes with high fidelity. To gain further insight into speech quality, we report the subjective MOS evaluations in Table 2. Overall, DiFlow-TTS consistently outperforms other methods across every MOS dimension, providing strong evidence of its well-balanced performance in generating natural and intelligible speech with high speaker similarity. It is worth noting that even though DiFlow-TTS ranks third on SIM-O (see Table 1), an embedding-space proxy that may penalize artifacts inaudible to humans, it best captures the perceptual identity cues (e.g., pitch, timbre, prosody) that human listeners value, indicating superior speaker faithfulness where it matters most. These results are especially notable given the model’s training data efficiency. For the other MOS metrics, the results are consistent with the findings in Table 1.

5.1.2. Model Size & Latency Analysis

Table 3 compares the model size and latency between DiFlow-TTS and the baselines for the 3-second audio prompt setting. The RTF metric, measured in seconds, shows that all baselines except OZSpeech and our method experience latency in the order of hundreds of milliseconds. To highlight efficiency, we further construct a smaller variant of DiFlow-TTS (denoted as *DiFlow-TTS-Small*) by reducing the number of attention heads ($12 \rightarrow 8$) and DiT layers ($12 \rightarrow 8$), resulting in a 122M-parameter model. This small variant achieves the best results in both speed and size. For comparison, OZSpeech, optimized for the 1-NFE setting, is expected to perform strongly in a strictly one-step regime. However, despite achieving the same RTF (0.03) as DiFlow-TTS-Small with 4 NFEs, it exhibits significantly lower performance

Table 3: Comparison of model size and latency. For **DiFlow-TTS**, we report practical low-latency configurations (NFE = 4, 16). The 128-NFE setting used in Table 1 achieves RTF = 0.39 (see Table 6). The #Params exclude the neural codec or vocoder component, which is non-trainable. The best and second best are **bold** and underlined, respectively.

Model	#Params ↓	NFE	RTF ↓	UTMOS ↑	WER ↓	SIM-O ↑	RMSE _{F0} ↓	RMSE _E ↓
VoiceCraft	830M	-	1.70	3.55	0.18	0.51	17.22	0.010
VALL-E	594M	-	0.86	3.68	0.19	0.40	21.66	0.020
NaturalSpeech 2	378M	200	1.66	2.38	0.09	0.31	15.62	0.020
F5-TTS	336M	32	0.26	3.72	0.09	<u>0.66</u>	12.66	0.010
OZSpeech	145M	1	0.03	3.15	0.05	0.40	11.96	0.010
MaskGCT	1.43B	50 + 45 [†]	0.46	3.83	0.09	0.67	14.33	0.007
DiFlow-TTS-Small (Ours)	122M	4	0.03	3.34	<u>0.06</u>	0.43	8.31	0.007
		16	0.05	3.89	0.05	0.45	8.58	<u>0.008</u>
DiFlow-TTS (Ours)	164M	4	0.03	3.31	0.05	0.44	<u>8.05</u>	0.007
		16	<u>0.07</u>	<u>3.86</u>	0.05	0.45	7.96	0.007

[†] MaskGCT is a two-stage system that first predicts masked semantic tokens, then uses them to infer masked acoustic tokens.

Table 4: Ablation study results showing the effect of removing each component from the DiFlow-TTS, with NFE set to 128. The best and second best are **bold** and underlined, respectively.

Model	UTMOS ↑	WER ↓	SIM-O ↑	F0		Energy	
				Accuracy ↑	RMSE ↓	Accuracy ↑	RMSE ↓
DiFlow-TTS (Ours)	<u>3.978</u>	0.048	0.454	0.884	7.972	0.735	0.007
- w/o Attribute Embedding	3.983	0.060	0.444	0.869	9.289	0.712	<u>0.008</u>
- w/o Speaker Embedding	3.902	<u>0.057</u>	0.378	0.681	20.868	0.615	0.010
- w/o Content Embedding	3.077	0.063	0.333	0.867	8.878	0.698	<u>0.008</u>
- w/o Multi-head Prediction	3.939	<u>0.057</u>	0.442	<u>0.876</u>	<u>8.474</u>	<u>0.726</u>	0.007

across all metrics except WER; meanwhile, DiFlow-TTS with 4 NFEs (RTF = 0.03) also attains performance comparable to our small variant. Furthermore, DiFlow-TTS-Small with 16 NFEs (RTF = 0.05) achieves competitive performance in naturalness, intelligibility, speaker similarity, and prosody error while being only marginally slower than OZSpeech (by 0.02s in RTF) yet $5.2\times$ to $34.0\times$ faster than the other baselines, with a model size $1.2\times$ to $11.7\times$ smaller than all baselines. The results of DiFlow-TTS with 16 NFEs further reinforce these findings, demonstrating a strong balance between model size, speed, and high-quality speech.

5.2. Ablation Studies and Analyses

5.2.1. Effect of Each Component

To assess the impact of each component in DiFlow-TTS, we perform an ablation study by systematically removing or modifying key elements: ① removing the attribute-type embeddings used to distinguish prosody, content, and acoustic streams; ② excluding the speaker embedding from the conditioning process (i.e., not injecting it into the DiT blocks); ③ disabling the use of content embeddings in the FDFD module; and ④ replacing the multi-head prediction architecture with a single-head prediction. As shown in Table 4, we observe a slight degradation in all metrics except UTMOS when the attribute-type embeddings are removed. This suggests that while these embeddings enhance overall fidelity and prosody modeling, they may introduce minor redundancies that subtly affect perceived naturalness. A more pronounced decline in speaker similarity and prosody-related metrics is observed when speaker embedding is excluded from the FDFD module. This highlights that prosody is not only content-dependent but also strongly influenced by speaker identity; without speaker conditioning, the FDFD module produces extraneous prosodic variations, resulting in reduced speaker adaptation and overall synthesis quality. When content

embeddings from the PCM branch are removed from FDFD, we observe substantial degradation across metrics related to naturalness and speaker similarity. This demonstrates the critical role of content embeddings in conditioning FDFD to generate appropriate prosody and support speaker adaptation. Lastly, replacing the multi-head prediction mechanism with a single-head alternative leads to minor performance drops across all metrics, indicating that the multi-head design enhances prediction diversity and robustness in prosody and acoustic modeling.

5.2.2. Attribute-type Embeddings

We conduct a finer-grained ablation to isolate the contribution of each attribute-specific embedding. The additional results are reported in Table 5. We observe that removing either the prosody attribute embedding or the acoustic attribute embedding leads to noticeable performance degradation, particularly in naturalness and metrics closely tied to these attributes. Attribute-type embeddings guide the model in distinguishing which attribute each token corresponds to when feeding them into the DiT. When an attribute embedding is removed, the model is forced to treat all attributes uniformly, which results in entanglement and reduced synthesis quality.

5.2.3. Effect of NFE

We investigate the impact of varying NFE from 1 to 128 on DiFlow-TTS performance to explore the trade-off between inference efficiency and synthesis quality, as presented in Table 6. Increasing NFE markedly improves UTMOS, indicating that the FDFD module benefits from additional refinement steps to generate more natural speech. In particular, performance stabilizes around 32 NFE, with optimal audio quality observed at 64 NFE, and only marginal improvements beyond this point. Although RTF naturally increases with NFE, the overall latency remains competitive (see Table 3). As reported in the Table 1, we show

Table 5: Ablation study on attribute-type embeddings.

Model	UTMOS $\uparrow$	WER $\downarrow$	SIM-O $\uparrow$	F0		Energy	
				Accuracy $\uparrow$	RMSE $\downarrow$	Accuracy $\uparrow$	RMSE $\downarrow$
DiFlow-TTS	3.978	0.048	0.454	0.884	7.972	0.735	0.007
- w/o Prosody Attribute	3.918	0.062	0.445	0.846	9.547	0.734	0.007
- w/o Acoustic Attribute	3.929	0.060	0.447	0.868	9.216	0.741	0.007

Table 6: Performance of UTMOS vs. NFE count.

NFE	RTF $\downarrow$	UTMOS $\uparrow$
1	0.022	2.904
2	0.025	2.908
4	0.031	3.313
8	0.043	3.698
16	0.066	3.864
32	0.112	3.923
64	0.207	3.958
128	0.394	3.978

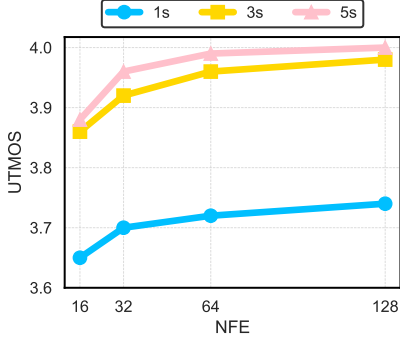


Figure 4: UTMOS vs. NFE for different prompt durations.

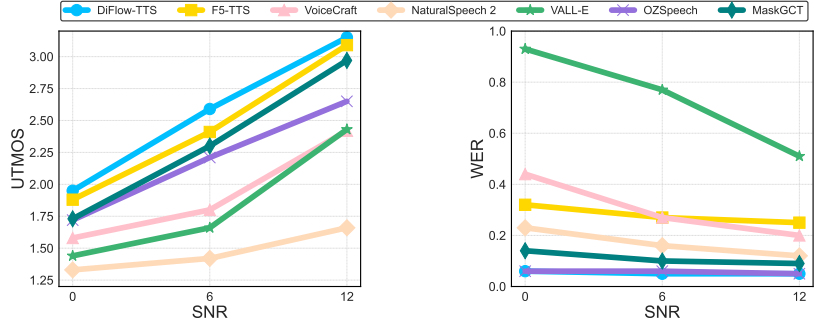


Figure 5: Model sensitivity under varying SNR levels in terms of UTMOS (left) and WER (right).

the **best performance at NFE = 128**; lower NFE values can be selected based on the user’s latency and quality requirements.

5.2.4. Prompt Duration Analysis

To gain further insight into model behavior, we report the relationship between UTMOS, which strongly correlates with human perceptual evaluations, and NFE across different prompt durations, as illustrated in Figure 4. In general, longer prompts lead to higher UTMOS scores, indicating improved reconstruction quality and a greater sensitivity of the model to prompt length. Additionally, increasing the NFE from 16 to 128 consistently improves performance for all prompt durations. In particular, the highest performance is achieved with a 5-second prompt and an NFE of 128.

5.2.5. Noisy Prompt Analysis

We analyze the robustness of DiFlow-TTS under noisy audio prompts using UTMOS and WER, a challenging scenario since most models are trained on clean speech. The noisy prompts are generated from the LibriSpeech test-clean set with additive noise augmentation. As shown in Figure 5, all models are highly sensitive to noise, showing sharp degradation in both UTMOS (left) and WER (right) as the prompt SNR decreases (see Table 1 for clean-prompt reference). DiFlow-TTS, however, consistently achieves the highest UTMOS across all noise levels, demonstrating its ability to synthesize high-fidelity speech under noisy conditions. For WER, it shows little to no degradation across SNR levels, a trend also observed in OZSpeech, while other baselines suffer significant performance drops.

6. Conclusion

In this paper, we introduce **DiFlow-TTS**, a novel zero-shot TTS framework that brings discrete flow matching into the domain of speech generation. By combining the PCM for accurate content

modeling with the FDFD that separately models prosody and acoustic attributes, DiFlow-TTS achieves strong performance in naturalness, intelligibility, prosody, and inference efficiency, while exhibiting limitations in speaker similarity as reflected by comprehensive objective and subjective evaluations. These results establish DiFlow-TTS as a compelling solution for efficient, high-fidelity zero-shot speech synthesis, well-suited to resource-constrained and latency-sensitive applications, and highlight discrete flow models as a promising direction for future generative speech research.

7. Limitations and Future Works

We demonstrated the viability of discrete flow matching for high-quality speech synthesis; however, our proposed method still exhibits several limitations. In particular, the current design **does not fully meet expectations in terms of voice cloning quality**; we acknowledge that our current strategy for speaker conditioning is not yet optimal. Since our framework relies on FAcCodec [10], which explicitly disentangles speech attributes and speaker identity, integrating it effectively requires additional mechanisms to inject speaker information. In contrast, codecs such as EnCodec [1] embed speaker information directly within their vector quantizers, enabling models to implicitly learn speaker characteristics.

In future work, we plan to extend our framework to **support alternative codec models** or to **replace global AdaLN conditioning with the in-context conditioning mechanism specifically designed for timbre embeddings**, allowing the model to capture local speaker characteristics more effectively. Developing more robust methods for reproducing speaker timbre from real-world audio in discrete settings, particularly in zero-shot TTS scenarios, remains an open research direction.

8. Generative AI Use Disclosure

The authors used Generative AI tools exclusively for manuscript refinement, including polishing and improving clarity, grammar, and sentence structure. All AI-assisted outputs were carefully reviewed and edited by the authors, who take full responsibility for the content of the current version.

9. References

- [1] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, “High fidelity neural audio compression,” *Transactions on Machine Learning Research*, 2023, featured Certification, Reproducibility Certification. [Online]. Available: <https://openreview.net/forum?id=ivCd8z8zR2>
- [2] N. Zeghidour, A. Luebs, A. Omran, J. Skoglund, and M. Tagliasacchi, “Soundstream: An end-to-end neural audio codec,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 30, pp. 495–507, 2021.
- [3] P. Peng, P.-Y. Huang, S.-W. Li, A. Mohamed, and D. Harwath, “VoiceCraft: Zero-shot speech editing and text-to-speech in the wild,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 12 442–12 462. [Online]. Available: <https://aclanthology.org/2024.acl-long.673>
- [4] L. Meng, L. Zhou, S. Liu, S. Chen, B. Han, S. Hu, Y. Liu, J. Li, S. Zhao, X. Wu, H. M. Meng, and F. Wei, “Autoregressive speech synthesis without vector quantization,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 1287–1300. [Online]. Available: <https://aclanthology.org/2025.acl-long.65/>
- [5] S. Chen, C. Wang, Y. Wu, Z. Zhang, L. Zhou, S. Liu, Z. Chen, Y. Liu, H. Wang, J. Li, L. He, S. Zhao, and F. Wei, “Neural codec language models are zero-shot text to speech synthesizers,” *IEEE Transactions on Audio, Speech and Language Processing*, pp. 1–15, 2025.
- [6] X. Wang, M. Jiang, Z. Ma, Z. Zhang, S. Liu, L. Li, Z. Liang, Q. Zheng, R. Wang, X. Feng *et al.*, “Spark-tts: An efficient llm-based text-to-speech model with single-stream decoupled speech tokens,” *arXiv preprint arXiv:2503.01710*, 2025.
- [7] D. Yang, J. Yu, H. Wang, W. Wang, C. Weng, Y. Zou, and D. Yu, “Diffsound: Discrete diffusion model for text-to-sound generation,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 31, pp. 1720–1733, 2023.
- [8] Z. Wu, Q. Li, S. Liu, and Q. Yang, “Dcetts: Discrete diffusion model with contrastive learning for text-to-speech generation,” in *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 11 336–11 340.
- [9] D. Yang, S. Liu, R. Huang, C. Weng, and H. Meng, “Instructtts: Modelling expressive tts in discrete latent space with natural language style prompt,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 32, pp. 2913–2925, 2024.
- [10] Z. Ju, Y. Wang, K. Shen, X. Tan, D. Xin, D. Yang, E. Liu, Y. Leng, K. Song, S. Tang, Z. Wu, T. Qin, X. Li, W. Ye, S. Zhang, J. Bian, L. He, J. Li, and S. Zhao, “NaturalSpeech 3: Zero-shot speech synthesis with factorized codec and diffusion models,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, Eds., vol. 235. PMLR, 21–27 Jul 2024, pp. 22 605–22 623. [Online]. Available: <https://proceedings.mlr.press/v235/ju24b.html>
- [11] K. Shen, Z. Ju, X. Tan, E. Liu, Y. Leng, L. He, T. Qin, sheng zhao, and J. Bian, “Naturalspeech 2: Latent diffusion models are natural and zero-shot speech and singing synthesizers,” in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=Rc7dAwVL3v>
- [12] K. Lee, D. W. Kim, J. Kim, S. Chung, and J. Cho, “DiTTo-TTS: Diffusion transformers for scalable text-to-speech without domain-specific factors,” in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=hQvX9MBowC>
- [13] Y. Qin, M. Madeira, D. Thanou, and P. Frossard, “Defog: Discrete flow matching for graph generation,” in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.
- [14] N. H. N. Hieu, N. S. Nguyen, H. N. Dang, T. Vo, T.-S. Hy, and V. Nguyen, “OZSpeech: One-step zero-shot speech synthesis with learned-prior-conditioned flow matching,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 21 500–21 517. [Online]. Available: <https://aclanthology.org/2025.acl-long.1043/>
- [15] S. Mehta, R. Tu, J. Beskow, E. Székely, and G. E. Henter, “Matchatts: A fast tts architecture with conditional flow matching,” in *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 11 341–11 345.
- [16] Y. Chen, Z. Niu, Z. Ma, K. Deng, C. Wang, J. JianZhao, K. Yu, and X. Chen, “F5-TTS: A fairytale that fakes fluent and faithful speech with flow matching,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 6255–6271. [Online]. Available: <https://aclanthology.org/2025.acl-long.313/>
- [17] P. Zheng, J. Wang, Y. Chang, Y. Yu, R. Ma, and Z. Wu, “Rethinking discrete tokens: Treating them as conditions for continuous autoregressive image synthesis,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 17 390–17 400.
- [18] I. Gat, T. Remez, N. Shaul, F. Kreuk, R. T. Q. Chen, G. Synnaeve, Y. Adi, and Y. Lipman, “Discrete flow matching,” in *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, Eds., vol. 37. Curran Associates, Inc., 2024, pp. 133 345–133 385. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/file/f0d629a734b56a642701bba7bc8bb3ed-Paper-Conference.pdf
- [19] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, “Neural discrete representation learning,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS’17. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 6309–6318.
- [20] A. Baevski, S. Schneider, and M. Auli, “vq-wav2vec: Self-supervised learning of discrete speech representations,” in *International Conference on Learning Representations*, 2020. [Online]. Available: <https://openreview.net/forum?id=rylwJxrYDS>
- [21] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, “Hubert: Self-supervised speech representation learning by masked prediction of hidden units,” *IEEE/ACM Trans. Audio, Speech and Lang. Proc.*, vol. 29, p. 3451–3460, Oct. 2021. [Online]. Available: <https://doi.org/10.1109/TASLP.2021.3122291>
- [22] C. Du, Y. Guo, F. Shen, Z. Liu, Z. Liang, X. Chen, S. Wang, H. Zhang, and K. Yu, “Unicats: A unified context-aware text-to-speech framework with contextual vq-diffusion and vocoding,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 16, pp. 17 924–17 932, Mar. 2024. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/29747>
- [23] D. Jia, Z. Chen, J. Chen, C. Du, J. Wu, J. Cong, X. Zhuang, C. Li, Z. Wei, Y. Wang, and Y. Wang, “DiTAR: Diffusion transformer autoregressive modeling for speech generation,” in *Forty-second International Conference on Machine Learning*, 2025. [Online]. Available: <https://openreview.net/forum?id=8tRtweTTwv>

- [24] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 6840–6851. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/4c5bcfec8584af0d967f1ab10179ca4b-Paper.pdf
- [25] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," in *International Conference on Learning Representations*, 2021. [Online]. Available: <https://openreview.net/forum?id=PxTIG12RRHS>
- [26] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=PqvMRDCJT9t>
- [27] X. Liu, C. Gong, and qiang liu, "Flow straight and fast: Learning to generate and transfer data with rectified flow," in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=XVjTT1nw5z>
- [28] W. Guan, Q. Su, H. Zhou, S. Miao, X. Xie, L. Li, and Q. Hong, "Reflow-ts: A rectified flow model for high-fidelity text-to-speech," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 10 501–10 505.
- [29] J. Yao, Y. Yuguang, Y. Pan, Z. Ning, J. Ye, H. Zhou, and L. Xie, "Stablevc: Style controllable zero-shot voice conversion with conditional flow matching," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, 2025, pp. 25 669–25 677.
- [30] J. Zuo, S. Ji, M. Fang, Z. Jiang, X. Cheng, Q. Yang, W. Liu, G. Zhang, Z. Tu, Y. Guo *et al.*, "Enhancing expressive voice conversion with discrete pitch-conditioned flow matching model," in *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2025, pp. 1–5.
- [31] A. Lou, C. Meng, and S. Ermon, "Discrete diffusion modeling by estimating the ratios of the data distribution," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, Eds., vol. 235. PMLR, 21–27 Jul 2024, pp. 32 819–32 848. [Online]. Available: <https://proceedings.mlr.press/v235/lou24a.html>
- [32] J. Shi, K. Han, Z. Wang, A. Doucet, and M. Titsias, "Simplified and generalized masked diffusion for discrete data," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. [Online]. Available: <https://openreview.net/forum?id=xqSOFHt4g>
- [33] A. Campbell, J. Yim, R. Barzilay, T. Rainforth, and T. S. Jaakkola, "Generative flows on discrete state-spaces: Enabling multimodal flows with applications to protein co-design," in *ICML*, 2024. [Online]. Available: <https://openreview.net/forum?id=kQwSbv0BR4>
- [34] K. Yi, K. Jamali, and S. H. Scheres, "All-atom inverse protein folding through discrete flow matching," in *Forty-second International Conference on Machine Learning*, 2025. [Online]. Available: <https://openreview.net/forum?id=8tQdwSCJmA>
- [35] H. Chang, H. Zhang, L. Jiang, C. Liu, and W. T. Freeman, "Maskgit: Masked generative image transformer," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2022, pp. 11 315–11 325.
- [36] M. Fuest, V. T. Hu, and B. Ommer, "Maskflow: Discrete flows for flexible and efficient long video generation," *arXiv preprint arXiv:2502.11234*, 2025.
- [37] Y. Ren, C. Hu, X. Tan, T. Qin, S. Zhao, Z. Zhao, and T.-Y. Liu, "Fastspeech 2: Fast and high-quality end-to-end text to speech," in *International Conference on Learning Representations*, 2021. [Online]. Available: <https://openreview.net/forum?id=piLPYqxtWuA>
- [38] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2023, pp. 4195–4205.
- [39] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "Roformer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, p. 127063, 2024.
- [40] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2019. [Online]. Available: <https://openreview.net/forum?id=Bkg6RiCqY7>
- [41] Y. Wang, H. Zhan, L. Liu, R. Zeng, H. Guo, J. Zheng, Q. Zhang, X. Zhang, S. Zhang, and Z. Wu, "MaskGCT: Zero-shot text-to-speech with masked generative codec transformer," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=ExuBFYtCQU>
- [42] G. Chen, S. Chai, G.-B. Wang, J. Du, W.-Q. Zhang, C. Weng, D. Su, D. Povey, J. Trmal, J. Zhang, M. Jin, S. Khudanpur, S. Watanabe, S. Zhao, W. Zou, X. Li, X. Yao, Y. Wang, Z. You, and Z. Yan, "Gigaspeech: An evolving, multi-domain asr corpus with 10,000 hours of transcribed audio," in *Interspeech 2021*, 2021, pp. 3670–3674.
- [43] H. He, Z. Shang, C. Wang, X. Li, Y. Gu, H. Hua, L. Liu, C. Yang, J. Li, P. Shi, Y. Wang, K. Chen, P. Zhang, and Z. Wu, "Emilia: An extensive, multilingual, and diverse speech dataset for large-scale speech generation," in *2024 IEEE Spoken Language Technology Workshop (SLT)*, 2024, pp. 885–890.
- [44] X. Zhang, L. Xue, Y. Gu, Y. Wang, J. Li, H. He, C. Wang, T. Song, X. Chen, Z. Fang, H. Chen, J. Zhang, T. Y. Tang, L. Zou, M. Wang, J. Han, K. Chen, H. Li, and Z. Wu, "Amphion: An open-source audio, music and speech generation toolkit," in *IEEE Spoken Language Technology Workshop, SLT 2024*, 2024.
- [45] H. Zen, V. Dang, R. Clark, Y. Zhang, R. J. Weiss, Y. Jia, Z. Chen, and Y. Wu, "Libritts: A corpus derived from librispeech for text-to-speech," in *Interspeech 2019*, 2019, pp. 1526–1530.
- [46] M. McAuliffe, M. Socolof, S. Mihuc, M. Wagner, and M. Sonderegger, "Montreal forced aligner: Trainable text-speech alignment using kald," in *Interspeech 2017*, 2017, pp. 498–502.
- [47] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, "Librispeech: An asr corpus based on public domain audio books," in *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2015, pp. 5206–5210.
- [48] T. Saeki, D. Xin, W. Nakata, T. Koriyama, S. Takamichi, and H. Saruwatari, "UTMOS: UTokyo-SaruLab System for VoiceMOS Challenge 2022," in *Interspeech 2022*, 2022, pp. 4521–4525.
- [49] S. Chen, C. Wang, Z. Chen, Y. Wu, S. Liu, Z. Chen, J. Li, N. Kanda, T. Yoshioka, X. Xiao *et al.*, "Wavlm: Large-scale self-supervised pre-training for full stack speech processing," *IEEE Journal of Selected Topics in Signal Processing*, vol. 16, no. 6, pp. 1505–1518, 2022.
- [50] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, "Hubert: Self-supervised speech representation learning by masked prediction of hidden units," *IEEE/ACM transactions on audio, speech, and language processing*, vol. 29, pp. 3451–3460, 2021.
- [51] J. Kahn, M. Rivière, W. Zheng, E. Kharitonov, Q. Xu, P. E. Mazaré, J. Karadayi, V. Liptchinsky, R. Collobert, C. Fuegen, T. Likhomanenko, G. Synnaeve, A. Joulin, A. Mohamed, and E. Dupoux, "Libri-light: A benchmark for asr with limited or no supervision," in *ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2020, pp. 7669–7673, <https://github.com/facebookresearch/libri-light>.
- [52] Z. Guo, Y. Leng, Y. Wu, S. Zhao, and X. Tan, "Prompttts: Controllable text-to-speech with text descriptions," 2022. [Online]. Available: <https://arxiv.org/abs/2211.12171>
- [53] S. Ji, J. Zuo, M. Fang, Z. Jiang, F. Chen, X. Duan, B. Huai, and Z. Zhao, "Textrolspeech: A text style control speech corpus with codec language text-to-speech models," in *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 10 301–10 305.